Quantitative Options Pricing Engine

Theoretical Foundations and Numerical Implementation

Sujesh Sheshadri

May 7, 2026

Abstract

This document outlines the theoretical foundations and numerical methodologies of a hybrid Python/C++ quantitative derivatives pricing engine. The project bridges the gap between pure analytical models and complex stochastic volatility environments. It details the implementation of the Black-Scholes-Merton analytical model, the Cox-Ross-Rubinstein Binomial Tree for American options, and the numerical solutions to the Heston Stochastic Volatility model via Monte Carlo simulation and 2D Explicit Finite Difference Methods. Furthermore, it outlines the calculation of risk exposures (The Greeks) using finite difference bumping.

Contents

1 Introduction to Options Pricing Theory

An option is a financial derivative that provides the holder the right, but not the obligation, to buy (Call) or sell (Put) an underlying asset at a specified strike price (K) on or before a maturity date (T).

- **European Options** can only be exercised exactly at maturity $t = T$.
- **American Options** can be exercised at any time $t \leq T$, creating an "early exercise premium" that requires complex numerical methods to evaluate.

The fundamental principle of modern derivatives pricing is *risk-neutral valuation*. It assumes that in an arbitrage-free market, the expected return of the underlying asset is the risk-free interest rate (r), and the option price is the discounted expected value of its terminal payoff.

2 The Black-Scholes-Merton (BSM) Model

Developed in 1973, the BSM model provides a closed-form analytical solution for pricing European options. It assumes that the underlying asset price follows a Geometric Brownian Motion (GBM) with a constant drift and, crucially, a *constant volatility* (σ).

2.1 Analytical Formulas

The price of a European Call (C) and Put (P) are given by:

$$C(S,t) = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2) \tag{1}$$

$$P(S,t) = Ke^{-rT}\Phi(-d_2) - S_0\Phi(-d_1) \tag{2}$$

Where $\Phi(x)$ is the cumulative standard normal distribution, and:

$$d_1 = \frac{\ln(S_0/K) + (r + \frac{\sigma^2}{2})T}{\sigma\sqrt{T}} \tag{3}$$

$$d_2 = d_1 - \sigma\sqrt{T} \tag{4}$$

2.2 Limitations

While mathematically elegant, the BSM model is restricted to European options. Furthermore, its core assumption—that volatility is constant—is empirically false. In reality, options with different strikes exhibit varying implied volatilities, a phenomenon known as the *Volatility Smile*.

3 The Binomial Pricing Model (CRR)

To evaluate American options, we use discrete-time numerical methods. The Cox-Ross-Rubinstein (CRR) Binomial Tree models the asset price over discrete time steps (Δt), moving either up by a factor u or down by a factor d.

3.1 Lattice Parameters

To match the variance of the Black-Scholes GBM, the CRR model defines:

$$u = e^{\sigma\sqrt{\Delta t}} \tag{5}$$

$$d = \frac{1}{u} = e^{-\sigma\sqrt{\Delta t}} \tag{6}$$

$$p = \frac{e^{r\Delta t} - d}{u - d} \tag{7}$$

Where p is the risk-neutral probability of an upward move.

3.2 Implementation and Early Exercise

The algorithm operates via backward induction. Starting at terminal nodes $t = T$, the payoff is computed as $\max(S_T - K, 0)$ for a Call. Sweeping backwards, the value at each node is the discounted expected value of the next period:

$$V_t = e^{-r\Delta t}\left[pV_{up} + (1-p)V_{down}\right] \tag{8}$$

For American options, the algorithm enforces an early exercise check at every node:

$$V_t = \max\left(V_t, \text{Intrinsic Value}\right) \tag{9}$$

4 The Heston Stochastic Volatility Model

To capture the empirical reality of the Volatility Smile, the Heston Model (1993) treats volatility itself as a random variable.

4.1 Stochastic Differential Equations (SDEs)

The model proposes a coupled system of SDEs, where variance (v_t) follows a Cox-Ingersoll-Ross (CIR) mean-reverting process:

$$dS_t = rS_t dt + \sqrt{v_t} S_t dW_t^S \tag{10}$$

$$dv_t = \kappa(\theta - v_t)dt + \sigma\sqrt{v_t} dW_t^v \tag{11}$$

The two Wiener processes are correlated by ρ, capturing the leverage effect (falling stock prices correspond with spiking volatility):

$$dW_t^S dW_t^v = \rho dt \tag{12}$$

Parameters: κ (mean reversion speed), θ (long-term variance), σ (volatility of volatility), and ρ (correlation).

4.2 The Heston PDE

Using Itô's Lemma and arbitrage-free portfolio arguments, the price of an option $U(S, v, t)$ satisfies the 2D Heston Partial Differential Equation:

$$\frac{\partial U}{\partial t} + \frac{1}{2}vS^2\frac{\partial^2 U}{\partial S^2} + \rho\sigma vS\frac{\partial^2 U}{\partial S \partial v} + \frac{1}{2}\sigma^2 v\frac{\partial^2 U}{\partial v^2} + rS\frac{\partial U}{\partial S} + \kappa(\theta - v)\frac{\partial U}{\partial v} - rU = 0 \tag{13}$$

5 Market Calibration via Python Optimization

Before the C++ engine can price options, the Heston model parameters must be calibrated to reflect current market realities. This is framed as a non-linear inverse optimization problem.

5.1 The Objective Function

Using the `yfinance` library, the Python orchestrator pulls a live options chain for a target ticker. The calibration routine seeks the parameter set $\Omega = \{\kappa, \theta, \sigma, \rho, v_0\}$ that minimizes the Sum of Squared Errors (SSE) between the real market prices (P_i^{Market}) and the model-implied prices (P_i^{Model}):

$$\min_{\Omega} \sum_{i=1}^{N} \left(P_i^{Market} - P_i^{Model}(\Omega; S, K, r, T)\right)^2 \tag{14}$$

5.2 SLSQP Optimization

Because the Heston parameters have strict mathematical bounds (e.g., variance must be positive $v_0 > 0$, correlation is bounded $-1 \leq \rho \leq 1$), the pipeline utilizes the Sequential Least Squares Programming (SLSQP) algorithm via SciPy. The algorithm navigates the parameter space while attempting to satisfy the Feller condition ($2\kappa\theta > \sigma^2$) to prevent variance from reaching zero. Once the global minimum is approximated, these parameters are serialized and passed to the C++ execution engine.

6 Software Architecture and Object-Oriented Design (OOP)

To ensure the high-performance C++ execution engines remain flexible and extensible, the project heavily utilizes Object-Oriented Programming (OOP) paradigms, specifically the Strategy Pattern.

6.1 The `PayOff` Class Hierarchy

Instead of hardcoding payoff logic—such as $\max(S - K, 0)$—deep inside the complex Monte Carlo or PDE loops, the architecture defines an abstract base class `PayOff` featuring a pure virtual functor:

```
class PayOff {
public:
    virtual double operator()(double Spot) const = 0;
    virtual ~PayOff() {}
};
```

6.2 Polymorphic Decoupling

Concrete classes such as `PayOffCall` and `PayOffPut` inherit from this base class and implement their specific financial logic. The numerical pricing functions (e.g., `HestonFDM_Explicit`) accept a constant reference to the base class (`const PayOff&`).

This polymorphism completely decouples the financial contract from the numerical solver. The exact same 2D C++ Finite Difference matrix solver can price Calls, Puts, or exotic derivatives simply by swapping the object passed to it at runtime, ensuring the codebase adheres to the Open-Closed Principle (open for extension, closed for modification).

7 Numerical Solutions for the Heston Model

7.1 Monte Carlo Simulation (Euler-Maruyama)

To solve the Heston model via Monte Carlo, we simulate the SDEs forward in time across thousands of parallel paths using the Euler-Maruyama discretization. To respect the correlation ρ between the independent random normal draws $Z_1, Z_2 \sim \mathcal{N}(0, 1)$, we use Cholesky decomposition:

$$Z_S = Z_1 \tag{15}$$

$$Z_v = \rho Z_1 + \sqrt{1 - \rho^2} Z_2 \tag{16}$$

Because discrete steps can push variance below zero, we apply Full Truncation ($v_t^+ = \max(v_t, 0)$):

$$S_{t+\Delta t} = S_t \exp\left(\left(r - 0.5 v_t^+\right) \Delta t + \sqrt{v_t^+ \Delta t} Z_S \right) \tag{17}$$

$$v_{t+\Delta t} = v_t + \kappa(\theta - v_t^+)\Delta t + \sigma \sqrt{v_t^+ \Delta t} Z_v \tag{18}$$

7.2 Explicit Finite Difference Method (FDM)

To price American options under Heston, we solve the PDE directly on a 2D grid (S vs. v). We approximate continuous derivatives using Central Difference operators:

$$\frac{\partial U}{\partial S} \approx \frac{U_{i+1,j} - U_{i-1,j}}{2\Delta S} \tag{19}$$

$$\frac{\partial^2 U}{\partial S^2} \approx \frac{U_{i+1,j} - 2U_{i,j} + U_{i-1,j}}{\Delta S^2} \tag{20}$$

By defining the PDE operator $\mathcal{L}(U)$, we sweep backwards in time explicitly:

$$U(t - \Delta t) = U(t) - \Delta t \cdot \mathcal{L}(U(t)) \tag{21}$$

The American early-exercise constraint is enforced at every grid node after each time step. Stability is maintained by strictly adhering to the Courant-Friedrichs-Lewy (CFL) condition, requiring a high number of time steps ($N_t \geq 5000$).

8 Risk Management (The Greeks)

Pricing is only half the quantitative equation; risk management is the other. The "Greeks" represent the partial derivatives of the option price with respect to various market parameters.

- **Delta ($\Delta = \frac{\partial U}{\partial S}$):** Directional risk.
- **Gamma ($\Gamma = \frac{\partial^2 U}{\partial S^2}$):** Convexity risk.
- **Vega ($\mathcal{V} = \frac{\partial U}{\partial v}$):** Volatility risk.
- **Theta ($\Theta = \frac{\partial U}{\partial t}$):** Time decay.
- **Rho ($\rho = \frac{\partial U}{\partial r}$):** Interest rate risk.

8.1 Numerical Calculation via Bumping

Instead of deriving analytical limits, institutional engines calculate Greeks numerically via Central Finite Difference Bumping. For example, Delta is computed by bumping the

asset price by a small ϵ:

$$\Delta \approx \frac{U(S_0 + \epsilon, \dots) - U(S_0 - \epsilon, \dots)}{2\epsilon} \tag{22}$$

By passing bumped parameters into the highly optimized C++ FDM engine, the Greeks automatically inherit the complex stochastic dynamics of the Heston framework, providing an accurate, real-world risk profile.

9 Conclusion

This project successfully synthesized mathematical theory with systems engineering. By treating model calibration as a non-linear inverse optimization problem in Python, and derivatives pricing as a coupled PDE system in C++, the architecture replicates the high-performance quantitative infrastructure utilized on modern trading desks.